

Open source

Presented by Dr. H. MESSABIH

Department of Computer Science
for 1st Year
Licence

Academic year 2025/2026



Chapter 2: Open Source Tools



What is software

Software is a set of instructions, data, or programs designed to operate computers and execute specific tasks.

There are mainly two categories of software:

- **Application Software:** These are programs designed to perform specific user-oriented tasks such as word processing, web browsing, or playing games. Applications rely on system software to function.
- **System Software:** This includes operating systems and other programs that manage and coordinate the computer hardware and provide a platform for applications to run.

Other types include programming software (tools used for developing software), middleware (which acts as a bridge between system software and applications), and driver software (which controls hardware devices).



What is software

In software development, two primary models of software exist:

- **Closed source software.**
- **Open source software**



What is Closed source software

Closed source software (also known as proprietary software), the public is not given access to the source code, so they can't see or modify it in any way. In short, it is referred as CSS.

- In closed source software the source code is protected.
- The only individual or organization who has created the software can only change it.
- The price of closed source software is high and users need to have valid and authenticated licenses to use the software.
- As is issues an authenticated license so it also put a lot restrictions on users based on usability and modification of software.

Some **examples of closed source software** are Skype, Google earth, Adobe Flash, Virtual Box, Adobe Reader, Microsoft office, Microsoft Windows, WinRAR, mac OS, Adobe Flash Player etc.



What is open source software

Open source software is software whose source code is made available to the public, allowing anyone to view, modify, and distribute it freely under licenses that comply with the principles of the Open Source Initiative. This approach encourages collaboration, transparency, and community-driven development, often resulting in robust and widely-used software solutions.



Difference between open source and free software

The difference between open source software and free software mainly lies in their philosophy and emphasis, though both share many common practices regarding source code availability and user freedoms.

Free Software

- Emphasizes the ethical and moral freedoms of users, focusing on the right to use, study, modify, and share software as a fundamental liberty.
- Advocated by the Free Software Foundation (FSF), it stresses user control over technology and collaboration within a community driven by ethical considerations.
- The term "free" refers to freedom (libre), not price, highlighting liberty rather than cost.
- Requires licenses that guarantee four essential freedoms: run the program, study and modify the code, redistribute copies, and distribute modified versions.



Difference between open source and free software

Open Source Software

- Focuses on the practical benefits of sharing source code, such as better quality, innovation, flexibility, and collaborative development efficiency.
- Promoted by the Open Source Initiative (OSI), it places less emphasis on ethical aspects and more on development models and outcomes.
- It also requires free access to source code and rights to modify and redistribute, but with a focus on development methodology and practical impact.



Difference between open source and free software

Similarities

- Both allow access to source code, allowing users to study, modify, and distribute software.
- Both rely on community collaboration and open licenses.

Free software is fundamentally about user freedoms as a social and ethical imperative, while open source software promotes open collaboration and practical advantages in software development. Though often interchangeable in practice, the distinction lies in philosophy and advocacy focus.



History of open source software

The history of open source software dates back to the early days of computing in the 1950s and 1960s when software was often freely distributed along with source code, especially with large-scale commercial computers like those from IBM. This early period was characterized by sharing and collaboration among users and developers.



History of open source software

However, during the 1970s, the trend shifted towards proprietary software, which restricted source code access. The free software movement in the mid-1980s, led by Richard Stallman with the introduction of the GNU Project, sought to restore the freedom to use, modify, and share software code, marking a major milestone. Stallman's goal was to "liberate everyone in cyberspace" by creating free software alternatives to proprietary ones.



History of open source software

The 1990s witnessed significant progress in open source development with the emergence of stable systems like GNU/Linux and 386 BSD. In 1998, the term "open source" was coined, and initiatives like the Mozilla Project launched when Netscape released its browser source code. The Open Source Initiative (OSI), founded by Eric Raymond and Bruce Perens in 1998, helped formalize open source principles and foster wider adoption. Open source software has since become foundational to modern computing, powering a vast array of technologies used globally by individuals, enterprises, and governments alike.



Key Characteristics

- Publicly accessible source code, enabling inspection and modification by anyone.
- Distribution permitted for both copies and derivative works, as long as the open license terms are followed.
- Typically developed collaboratively by a community of individual contributors or organizations, rather than being controlled solely by a single company.



Advantages of Open Source Software

- **Cost-Effectiveness:** Most open source software is free to download and use, eliminating costly licensing fees and reducing overall ownership costs. This can be particularly beneficial for startups and small businesses.
- **Flexibility and Customization:** Access to the source code allows users to modify and tailor the software to fit specific workflows or requirements, enabling innovation and adaptation without dependence on vendors.
- **Community and Collaboration:** Open source projects benefit from active global communities that contribute to rapid development, bug fixes, feature additions, and support, fostering continuous improvement and reliability.



Advantages of Open Source Software

- **Transparency and Security:** Open inspection of source code allows quicker identification and patching of security vulnerabilities. This transparency helps maintain high security standards through community vigilance.
- **Avoidance of Vendor Lock-In:** Users have full control over the software, reducing dependency on a single vendor and mitigating risks of discontinued support or restrictive contracts.



Disadvantages of Open Source Software

- **Security Risks:** While transparency helps with security, it also means vulnerabilities can be exploited quickly if patches are not applied timely. Security relies heavily on the community's responsiveness.
- **Limited Dedicated Support:** Unlike proprietary software, open source may lack formal customer support, requiring reliance on community forums or in-house technical expertise.
- **Complexity and Technical Expertise:** Installation, configuration, and maintenance may require specialized technical skills, which could be a barrier for organizations without dedicated IT staff.



Disadvantages of Open Source Software

- **Potential Hidden Costs:** Although OSS is free, there can be costs related to implementation, customization, training, and ongoing maintenance that are sometimes underestimated.
- **Compatibility and Integration Issues:** OSS may sometimes face challenges integrating with existing proprietary systems or software, leading to additional resource needs.

In summary, open source software provides significant benefits like cost savings, customization freedom, and collaborative innovation, but it also poses challenges such as security management and the need for technical expertise. Organizations should weigh these factors against their technical capabilities and business needs to make an informed decision.



What is a license in open source software?

An open source software license is a legal agreement that defines the rules and conditions under which the software's source code can be used, modified, and distributed. It serves to protect the intellectual property rights of the original creators while granting users freedoms that align with open source principles, such as the ability to access, change, and share the code.

Open source licenses provide the legal framework that enables collaboration and innovation in software development while maintaining the balance of rights and obligations between authors and users.



What is a license in open source software?

There are two main types of open source licenses:

- **Permissive Licenses:** These allow users to freely use, modify, and distribute the software, including incorporation into proprietary software, often with minimal conditions like attribution. Examples include the MIT License, BSD License, and Apache License.
- **Copyleft Licenses:** These require that any modified versions or derivative works be distributed under the same license terms, ensuring that the software remains open and free. The GNU General Public License (GPL) is a well-known copyleft license.



Open source software types

Open source software can be categorized into many types based on its applications and functionality. Some common types of open source software include:

- **Operating Systems:** Open source OS like Linux distributions that manage hardware and provide a platform for applications.
- **Office Software Suites:** Examples include LibreOffice, providing word processing, spreadsheets, and presentation tools.
- **Development Tools:** IDEs, compilers, editors, and frameworks that help developers create software (e.g., Eclipse, Visual Studio Code).
- **Content Management Systems (CMS):** Software like Drupal and WordPress for building and managing websites.
- **Database Management Systems:** Open source databases such as MySQL and PostgreSQL.



Open source software types

- **Network and Security Software:** Tools like firewalls, antivirus, and VPN clients.
- **Multimedia Software:** Audio and video players, editors, and codecs.
- **Web Browsers:** Open source browsers such as Mozilla Firefox.
- **Utilities and System Software:** File managers, system monitors, and backup software.

The key characteristic of all these types is that their source code is freely available for use, modification, and distribution under open source licenses.



Example of open source software

Examples of popular open source software include:

- **Linux:** A widely-used open source operating system kernel that powers many servers, desktops, and devices globally.
- **Mozilla Firefox:** A free, open source web browser known for privacy and customization features.
- **LibreOffice:** An open source office suite providing word processing, spreadsheets, presentations, and more as a free alternative to proprietary suites.
- **GIMP:** A powerful open source image editor used for photo retouching and graphic design.



Example of open source software

- **VLC Media Player:** An open source multimedia player that supports a wide variety of audio and video formats.
- **Audacity:** Popular open source audio recording and editing software.
- **Kali Linux:** Specialized open source Linux distribution for penetration testing and security audits.
- **OpenSSH:** Open source implementation of the Secure Shell protocol for secure network communications.

These examples represent various categories like operating systems, productivity, multimedia, and security tools, illustrating the broad impact and utility of open source software across different domains.



Companies supporting open source software

Major companies supporting open source software include:

- **IBM:** A long-time advocate and contributor, IBM supports open source projects such as the Linux kernel, Apache web server, and Eclipse IDE. It also invests in emerging technologies like blockchain through open source initiatives.
- **Google:** A significant contributor with projects like Android, Chrome, Kubernetes, and TensorFlow. Google actively supports open source through infrastructure, funding, and community engagement.
- **Microsoft:** Once a notable opponent of open source, Microsoft has embraced it by releasing open source projects like .NET Core and Visual Studio Code, contributing to the Linux kernel, and supporting cloud-based open source technologies on Azure.



A **development environment** is a workspace equipped with the processes, tools, configurations, and resources necessary for developers to write, test, debug, and refine code before it is deployed to production. It serves as a controlled setting separate from the live environment, allowing developers to experiment, identify, and fix issues without impacting end users.



Components of a Development Environment

A typical **development environment** includes

- hardware (computers or servers),
- the operating system,
- programming languages,
- development tools such as integrated development environments (IDEs),
- version control systems to manage source code changes,
- and build automation tools that compile and package the software.

It may also include testing frameworks, databases, and network infrastructure to connect components.



Types of Development Environments

- **Local Development Environment:** Developers work on their own machines using IDEs or editors for coding, testing, and debugging.
- **Integrated Development Environment (IDE):** Software that combines a code editor, compiler/interpreter, debugger, and project management tools in a single interface.
- **Cloud-Based Development Environment:** Enables developers to code and test in a virtualized setting accessible via the internet, supporting remote collaboration.
- **Others:** Command line environments, mobile development environments, game development environments tailored to specific needs.



Purpose and Benefits of Development Environments

Development environment is essential for enabling multiple developers to

- collaborate efficiently without overwriting each other's code,
- fostering experimentation and learning,
- and ensuring that software quality is verified before deployment.



Code editor

A **code editor** is a specialized software application designed primarily for writing and editing programming code. It is more advanced than a basic text editor because it offers features that help programmers write code more efficiently and accurately.



Code editor

There are two main types of code editors:

- **lightweight editors**, fast and simple, suitable for quick edits or smaller projects (examples include Sublime Text, Visual Studio Code, Atom).
- **integrated development environments (IDEs)**, which combine a code editor with additional development tools such as debuggers and version control. More comprehensive with tools beyond editing, suited for larger or more complex projects (examples include IntelliJ IDEA, Eclipse).



Key Features of Code Editors

- Syntax highlighting: Different elements of code (keywords, variables, strings) are displayed in distinct colors to improve readability.
- Code suggestions and auto-completion: These features help speed up coding by suggesting or completing lines or blocks of code.
- Debugging support: Some editors offer built-in or extensible debugging tools.
- Search and replace: Easily find and modify text within files or projects.
- Integration with version control systems (e.g., Git).
- Customizable themes and user interface enhancements.
- Support for multiple programming languages and frameworks with extensions or plugins.



Usage Context

Code editors are essential tools where programmers spend most of their time coding. They replace basic text editors like Notepad or TextEdit with specialized environments better suited for programming tasks. Code editors can be used offline as downloadable applications or online through browser-based platforms.



Linux

Linux is a powerful and popular environment for software development due to its open-source nature, stability, security, and extensive support for programming languages and tools. As a development environment, Linux provides a rich ecosystem of programming libraries, compilers, debuggers, and editors, closely aligned with UNIX standards, which are favored in many programming and server environments.



Development Environments

Key Advantages of Linux

- **Open Source:** Linux is free to use and modify, allowing developers full control over their environment.
- **Stability and Performance:** Known for high stability and efficient use of system resources, making it ideal for running development workloads.
- **Security:** Built-in security features and regular updates help maintain a secure development platform.
- **Wide Language Support:** Supports many programming languages like C, C++, Python, Java, Ruby, Node.js, and more.
- **Customizability:** Developers can tailor the system extensively to suit their specific workflow and preferences.
- **Tools and Utilities:** Comes with numerous development tools such as package managers, version control systems (Git), build automation tools (make), and powerful editors (Vim, Emacs).

Typical Development Setup on Linux

Developers often choose a suitable Linux distribution (e.g., Ubuntu, Fedora) and install their preferred programming languages, text editors or IDEs, version control systems, and other utilities. The terminal is a critical part of the workflow, allowing script execution, system navigation, and tool integration.



Usage Context

Linux is widely used for various kinds of development including system programming, web development, embedded systems, and scientific computing. The close alignment with UNIX makes it essential in environments where software must run on servers or cloud infrastructure.

